Q1 (a) Adam Optimizer and Bias Terms:

Adam keeps running averages of gradients m_t and squared gradients v_t.

Update equations:

m_t = beta1 * m_{t-1} + (1 - beta1) * g_t

v_t = beta2 * v_{t-1} + (1 - beta2) * g_t^2

Since m_0 is initialized to 0, m_t starts biased toward 0 in early iterations.

Bias correction divides by (1 - beta1^t) to correct this zero-bias.

Unbiased estimators: m_hat = m_t / (1 - beta1^t) and v_hat = v_t / (1 - beta2^t).

Parameter update: theta_{t+1} = theta_t - alpha * m_hat / (sqrt(v_hat) + eps).

Q1 (b) Momentum vs Adam in Ravines:

In ravines, loss is steep in one direction and flat along the other.

SGD oscillates back and forth across walls without moving forward.

NAG calculates gradient at look-ahead position to slow down before overshooting.

Adam adjusts the step size for each coordinate using sqrt(v_t), which scales down updates along steep dimensions and moves faster along gentle slopes.

Q2 (a) Transformer Scaled Dot Product Attention:

Attention(Q, K, V) = softmax(Q * K^T / sqrt(d_k)) * V.

Why scale by 1 / sqrt(d_k)?

When dimension d_k is large, the dot product between Q and K becomes very large.

This is because the sum of d_k terms adds up to variance proportional to d_k.

Large dot products push softmax inputs into saturated regions where gradients become tiny.

Dividing by sqrt(d_k) normalizes the variance back to 1, preventing vanishing gradients.

Q2 (b) Multi-Head Attention and Complexity:

Instead of 1 attention calculation, Multi-Head attention projects Q, K, V into h different heads.

Each head attends to different features or positions in the text.

Complexity: Self-attention is O(n^2 * d) while RNN is O(n * d^2).

Self-attention allows parallel computing because all words are processed at once.

Q3 (a) Normalization Techniques:

BatchNorm normalizes across batch items. Requires large batch sizes and running stats.

LayerNorm normalizes across feature dimension per individual sample.

LayerNorm works well for NLP and Transformers because sequence lengths vary.

Q3 (b) Inverted Dropout:

Inverted dropout divides activations by (1 - p) during training: a = a / (1 - p).

This preserves the expected value during training, so at test time no scaling is needed.

Q4 (a) ResNet Skip Connections & Gradient Flow:

ResNet layer equation: x_{l+1} = x_l + F(x_l).

Gradient: dE/dx_l = dE/dx_L * (I + d/dx_l sum F(x_i)).

The identity matrix I ensures that gradient flows straight back without decaying.

Even if F(x) weights produce small gradients, the I term keeps gradient alive.

Q4 (b) Pre-Activation puts BN and ReLU before the weight layer instead of after addition.